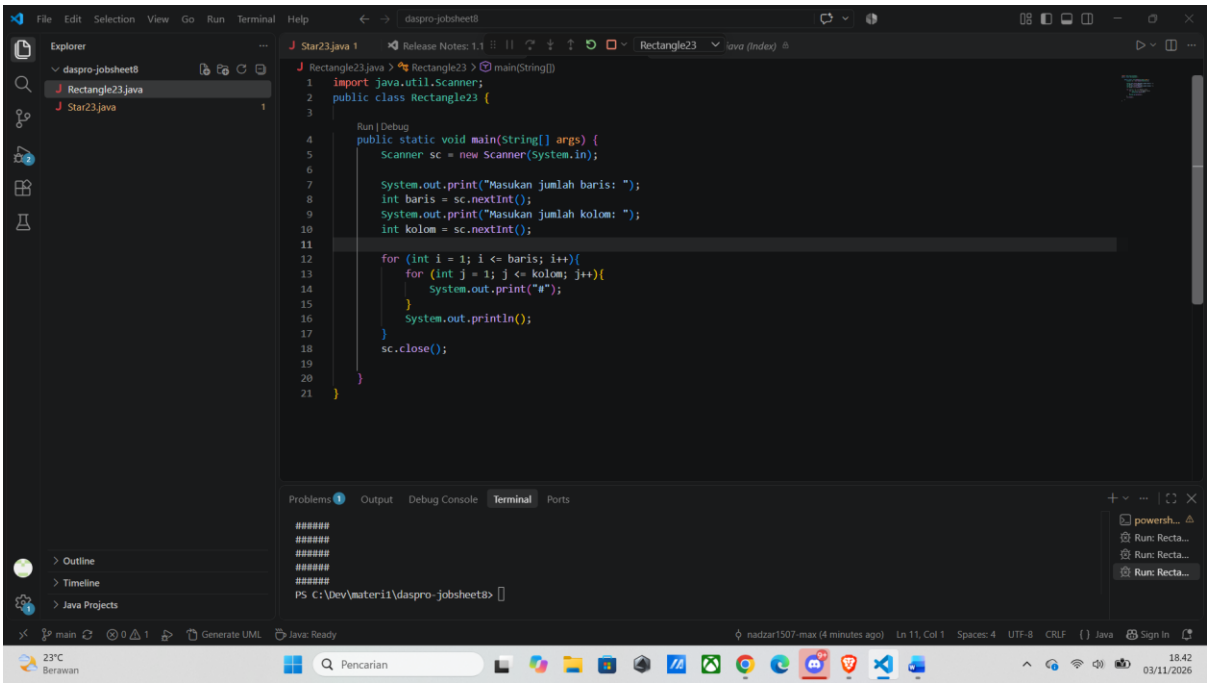
Nama : Muhammad nadzar al ghifary
Kelas : 1H
No. Presensi : 23

Pertanyaan 2.1

1. Jawaban: Karakter * yang tercetak akan menjadi 6 buah.
   Penjelasan: Karena batas kondisinya menggunakan <= ($i \leq n$), perulangan akan dieksekusi saat $i = 0,1,2,3,4,5$. Totalnya ada 6 kali iterasi, sehingga bintang yang dicetak bertambah satu dibanding saat $imulainyadari1$.
2. Jawaban: Perulangan akan dijalankan sebanyak 4 kali ($i = 1,2,3,4$).
   Penjelasan: Hasilnya berbeda karena tanda < tidak menyertakan angka n-nya (5). Jadi saat $i mencapai angka 5, kondisi $5 < 5$ bernilai *false*, sehingga perulangan langsung berhenti sebelum mencetak bintang yang kelima.
3.
```
for (int i = 1 ; i <= n; i+=2 ){
    System.out.print("*");
}
```
   Modifikasi Kode untuk Bilangan Ganjil: Supaya mencetak bilangan ganjil positif yang tidak melebihi n, kita cukup mengubah nilai awal i menjadi 1 dan menjaga *increment*-nya tetap bertambah 2 ($i+= 2$).
4. Jawaban: Karena arah perubahannya salah. Nilai awal i adalah 1 (yang sudah kurang dari atau sama dengan n), dan alih-alih bertambah mendekati n (lewat $i++$), nilai i malah dikurangi (lewat $i--$) menjadi $0, -1, -2, \ldots$ dan seterusnya. Akibatnya, kondisi $i \leq n$ akan selamanya bernilai benar (true), yang memicu *infinite loop* (perulangan tak berhingga).

Pertanyaan 2.2

1. Variabel i: Berfungsi sebagai pengendali atau penghitung pada perulangan luar (*outer loop*) yang bertugas mengatur jumlah baris yang akan dicetak ke bawah.
   Variabel j: Berfungsi sebagai pengendali atau penghitung pada perulangan dalam (*inner loop*) yang bertugas mengatur jumlah kolom (bintang ke samping) pada setiap barisnya.
2. Outer loop: Dijalankan sebanyak 4 kali (mengikuti jumlah baris).
   Inner loop: Dijalankan sebanyak 28 kali secara total (karena untuk setiap 1 baris, inner loop berjalan 7 kali ke samping $4 \times 7 = 28$).
   Perintah print("*"): Dijalankan sebanyak 28 kali (sama dengan jumlah eksekusi inner loop karena mencetak satu bintang tiap satu iterasi).
3. Hasil bentuk cetakannya: Menjadi bentuk persegi panjang vertikal/berdiri, bukan mendatar lagi.
   Jumlah total baris: Menjadi 6 baris (mengikuti nilai kolom).
   Jumlah bintang per baris: Menjadi 3 bintang per baris (mengikuti nilai baris).
4. jika perintah System.out.println(); dihapus, maka seluruh karakter bintang akan tercetak berderet memanjang ke samping dalam satu baris tunggal. Program tidak akan memberikan perintah turun ke baris baru setelah satu baris kolom selesai dicetak, sehingga bentuk persegi panjangnya akan rusak dan bergabung menjadi satu garis lurus panjang.

5. 

Pertanyaan 2.3

1. Ketidaksesuaian output awal:
   Semua karakter bintang tercetak menumpuk/menyamping dalam satu baris karena tidak ada perintah untuk pindah baris (System.out.println()).
   Jumlah baris yang dihasilkan tidak sesuai dengan nilai *n* karena inisialisasi i = 0 dan kondisi i <= n membuat perulangan berjalan kelebihan satu kali (menghasilkan baris kosong di awal).

   Minimal dua perbaikan yang diperlukan:
   Menambahkan perintah System.out.println(); di dalam *outer loop* (setelah *inner loop* selesai) agar kursor turun ke baris baru setiap kali satu baris bintang tercetak.
   Mengubah inisialisasi variabel i dari 0 menjadi 1 (int i = 1;) agar jumlah barisnya pas sebanyak *n*.
2. -Peran variabel `i`: Berperan sebagai pengendali pada perulangan luar (*outer loop*) untuk mengatur jumlah baris yang akan dicetak ke bawah.
   -Peran variabel `j`: Berperan sebagai pengendali pada perulangan dalam (*inner loop*) untuk menghitung jumlah karakter bintang (`*`) yang dicetak menyamping pada setiap barisnya.
   Mengapa `j` harus di-reset ke 0? Nilai `j` harus dikembalikan ke `0` di setiap awal iterasi *outer loop* agar hitungan bintang pada baris baru selalu dimulai dari awal secara konsisten.
   Apa yang terjadi jika `j` tidak di-reset? Nilai `j` akan terus bertambah dari baris-baris sebelumnya dan tidak kembali ke nol, sehingga jumlah bintang pada baris berikutnya akan menjadi berlebih atau salah total.

3. Inisialisasi: Mengubah nilai awal `i` dari `1` menjadi `n` (`i = n;`) agar perulangan dimulai dari baris dengan jumlah bintang terbanyak.

Kondisi: Mengubah kondisi dari `i <= n` menjadi `i >= 1` agar perulangan terus berjalan selama `i` belum mencapai angka 1 (berhenti saat habis).
Perubahan nilai (update): Mengubah *update* nilai dari penambahan (`i++`) menjadi pengurangan (`i--`) agar jumlah maksimum bintang pada setiap barisnya berkurang satu per satu secara bertahap hingga membentuk ujung yang lancip di bawah.

Pertanyaan 2.4

1. TotalNilai harus di-reset ke 0 pada setiap awal perulangan kelompok (while) agar akumulasi nilai hanya menghitung nilai penilai untuk kelompok yang sedang berjalan saja, tidak tercampur dengan nilai dari kelompok sebelumnya. RataTertinggi tidak boleh di-reset karena bertindak sebagai variabel pembanding global yang harus menyimpan rekor nilai rata-rata tertinggi dari kelompok-kelompok sebelumnya agar bisa terus dibandingkan dengan kelompok berikutnya hingga proses selesai.
2. Jumlah eksekusi badan perulangan FOR untuk 3 kelompok dan 2 penilai: Perulangan FOR (penilai) dijalankan sebanyak 6 kali secara keseluruhan (3 kelompok × 2 penilai).
3. Akibat jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian bilangan bulat: Hasil perhitungan rata-rata akan mengalami pembulatan ke bawah (kehilangan nilai desimalnya/patahan angka di belakang koma) karena operasi pembagian sesama integer di Java menghasilkan bilangan bulat. Contoh: Jika suatu kelompok mendapat total nilai 135 dari 2 penilai, seharusnya rata-rata bernilai 67.5. Namun, jika tipe datanya int, hasilnya akan langsung dipotong menjadi 67.
4. Alasan rataTertinggi diberi nilai awal -1, bukan 0: Nilai awal -1 diberikan agar kelompok pertama yang masuk (meskipun rata-ratanya kecil atau di atas 0) pasti akan langsung terpilih sebagai rataTertinggi pertama. Contoh input yang membuat salah jika nilai awal 0: Jika seluruh kelompok yang diuji ternyata memiliki rata-rata 0, maka inisialisasi 0 tidak akan pernah bisa diperbarui oleh kelompok pertama karena nilainya tidak lebih besar dari 0.
5. while (Luar): Berperan sebagai perulangan untuk memproses setiap kelompok satu per satu hingga mencapai jumlah kelompok yang ditentukan.

   for (Tengah): Berperan untuk mengontrol dan melayani proses input nilai dari setiap penilai untuk kelompok yang sedang aktif.

   do-while (Dalam): Berperan untuk melakukan validasi input (mengulang permintaan masukkan nilai) apabila penilai memasukkan angka di luar rentang yang diizinkan (yaitu di bawah 0 atau di atas 100).